



Article

Lightweight Deep Learning with Intra-Class Half-Mixing and Geometric Augmentation for Imbalanced Oil Palm Fresh Fruit Bunch Ripeness Classification

Hadee Madadum ^{1,2,*} , Fazal E. Nasir ³ and Kanjana Haruehansapong ¹

¹ School of Informatics, Walailak University, Nakhon Si Thammarat 80160, Thailand; hkanjana@wu.ac.th

² Informatics Innovation Center of Excellence (IICE), Walailak University, Nakhon Si Thammarat 80160, Thailand

³ Department of Biological and Agricultural Engineering, North Carolina State University, Raleigh, NC 27695, USA; fnasir@ncsu.edu

* Correspondence: hadeema@wu.ac.th

Abstract

The precision of oil palm fresh fruit bunch (FFB) ripeness classification directly determines the extraction efficiency and chemical quality of the resulting crude palm oil (CPO). Traditional manual inspection at reception ramps remains labour-intensive, time-consuming, and subjective. To address these issues, this study presents a lightweight deep learning framework for automated oil palm FFB ripeness classification trained on a field-collected dataset of 857 images covering four ripeness classes (Over-ripe, Ripe, Under-ripe, and Unripe). Six lightweight classification backbones are evaluated, including MobileNetV2, EfficientNetV2B0/B1, and YOLO variants. An intra-class half-mixing augmentation with geometric transformation is proposed to address minority-class imbalance. Overall, YOLOv8n-cls achieved the highest accuracy (95.6%), followed by EfficientNetV2B0/B1, YOLO11n-cls, YOLO26n-cls, and MobileNetV2, respectively. In addition, all YOLO-family models achieved a recall score of 1.00 for the minority class while obtaining the highest F_1 scores for the other classes. The experimental results suggest that the proposed augmentation method enables lightweight deep learning models to achieve promising classification performance on an imbalanced field-collected FFB dataset while improving minority-class detection.

Keywords: transfer learning; fruit ripeness classification; minority-class detection; computer vision; precision agriculture



Academic Editor: Fernando Ferrari Putti

Received: 28 May 2026

Revised: 17 July 2026

Accepted: 17 July 2026

Published: 20 July 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Oil palm is a major economic crop for Thailand, which ranks as the 3rd largest palm oil producer in the world [1]. Palm oil is an important industrial raw material used for both human consumption, particularly household cooking oil, and industrial manufacturing applications. Oil palm cultivation is widely preferred by farmers in Thailand, particularly in the southern region, due to the high economic return compared with other agricultural activities. However, determining the optimal harvest time remains a major challenge for farmers. The quality and quantity of crude palm oil (CPO) extracted from fresh fruit bunches (FFB) depend strongly on harvest timing. Under-ripe bunches produce oil with high free fatty acid (FFA) content, while over-ripe bunches increase FFA formation during transport, which reduces extraction efficiency and market value [2,3]. Therefore,

the accuracy of ripeness grading upon receipt is critical to the processing quality and profitability of the plantation. Conventionally, FFB ripeness grading is performed by trained inspectors who visually evaluate FFBs based on fruit color, surface texture, and loose fruitlet count. This manual process is inherently subjective, time-consuming, and variable between inspectors.

Therefore, automated machine vision systems have been proposed as efficient alternatives to manual FFB grading. Ref. [4] reviewed non-destructive FFB ripeness classification methods, including spectroscopy, thermal imaging, LiDAR, and computer vision, and concluded that computer vision is the most practical approach due to its cost efficiency and compatibility with existing mill infrastructure. Recently, Convolutional neural networks (CNNs) have been demonstrated as computer vision-based approaches that achieve high performance in image classification. In modern agricultural applications, CNN-based approaches have been widely adopted for fruit ripeness classification, including banana ripeness classification [5], tomato ripeness classification [6], and general fruit ripeness assessment [7]. In addition, the application of deep learning to oil palm FFB ripeness detection has been systematically reviewed by [8]. They surveyed 51 studies covering 11 distinct approaches and concluded that computer vision combined with deep learning is the most feasible method for field deployment.

However, to deliver in the real-world ripeness grading system, it requires not only high accuracy but also efficient computation. Mill reception ramps, especially those operated by smallholder cooperatives, are often remote facilities with limited computational infrastructure and no reliable access to cloud services. The classification system must therefore operate on local edge-computing hardware, requiring limited memory, low computational cost, and short inference time to support near-real-time FFB ripeness grading under field conditions. A recent review of deep learning for fruit image analysis by [9] similarly identified lightweight model design and limited labelled data as two major challenges for practical agricultural deployment.

Several prior works have explored lightweight CNN architecture to develop an automated grading system under constrained resources. MobileNet and EfficientNet-based models were evaluated in [10]. MobileNetV3 was also demonstrated for a four-class FFB dataset and achieved 91.5% accuracy in [11]. In addition, a compact custom CNN was presented in [12] for FFB classification with an accuracy of 81.25%. These results show that lightweight CNN architectures can provide feasible performance for practical FFB grading systems. More recently, the You Only Look Once (YOLO) framework has been adapted to FFB classification tasks [13]. Although YOLO variants were originally designed for object detection [14], they can deliver competitive accuracy with low inference time on small devices. Therefore, YOLO-based approaches are potential models for real-time deployment in limited operating environments, including palm oil reception ramp stations [15–18].

Class imbalance is another critical challenge in real-world FFB classification. Under practical intake conditions, especially at the ramp reception, ripe and under-ripe bunches often occur more frequently than other classes because they are ready for oil production. As a result, field-collected FFB datasets are generally imbalanced. Nevertheless, rarely observed bunches, such as the over-ripe grade, are still important for quality control [3,19]. Low classification performance on minority ripeness classes may then decrease grading reliability and affect downstream processing quality.

Several techniques have been proposed to resolve imbalanced image datasets, including a classic augmentation with photometric and geometric transformations [20], synthetic oversampling [21], and generative augmentation [22]. However, these existing methods may not always provide sufficient minority class diversity for field-based FFB images where ripeness related visual patterns are influenced by illumination variation, viewing

angle, occlusion, background complexity and natural color differences. In addition, a recent survey of class imbalance methods in agricultural image datasets [23] notes that data-level and algorithm-level approaches remain under-investigated in combination.

For this reason, this study aims to develop a lightweight FFB ripeness classification framework designed specifically for resource-limited palm ramp reception environments. The proposed system utilizes field-collected FFB images for training. A class-aware augmentation strategy is proposed to address class imbalance in real field-collected dataset. Based on the research problem described above, this study hypothesizes that combining intra-class mixing with classic geometric transformation can improve minority-class recognition in imbalanced oil palm FFB ripeness classification while maintaining the computational efficiency required for lightweight model development under resource-constrained deployment conditions. The specific objectives are presented as follows:

- To establish a four-class oil palm FFB dataset collected under unconstrained mill-ramp conditions, with natural illumination variation, multiple viewing angles, and ripeness labels validated by trained industry grading experts.
- To propose and evaluate a class-aware intra-class half-mixing augmentation strategy combined with geometric transformation to improve minority-class recognition in imbalanced field-collected FFB data.
- To evaluate and compare multiple lightweight deep learning architectures under different augmentation strategies, considering both classification performance and computational characteristics, in order to identify promising backbones for future deployment on resource-constrained edge platforms at mill reception stations.

2. Materials and Methods

2.1. Dataset

2.1.1. Data Collection and Production

A dataset of oil palm FFB images was assembled under varied field illumination conditions and bunch orientations representative of plantation harvest conditions. All images were captured at a fixed distance of 1–1.5 m using smartphone camera (iPhone 13 Pro) to simulate conditions representative of data acquisition by the mill reception ramp. We employed three collectors who captured the palm bunch images equally to reduce the bias of collecting images. Images were captured at the center of each palm bunch and only one main palm bunch per image. To incorporate variability in lighting conditions, images were collected at different times of day including morning at 10.00–12.00 and evening at 16.00–18.00 over one week. The dataset was collected from an oil palm ramp located in Khlong Thom District, Krabi Province, Thailand at the geographic coordinates of 7.892781° N, 99.115334° E.

After the data collection process, the annotation task was performed by three trained FFB grading experts who were workers from the palm ramp. Each image was reviewed by the three evaluators based on their practical grading experience. When disagreement occurred among the evaluators, the image was re-examined and discussed collectively, and the final ground-truth label was assigned through consensus discussion.

The original independent annotations from each grader were not retained after the consensus labeling process. Consequently, inter-rater agreement statistics, such as Cohen's Kappa, could not be calculated retrospectively. Although the consensus labeling procedure was adopted to establish the final ground-truth labels, the absence of the original independent annotation records prevented quantitative validation of annotation reliability.

The dataset comprised 857 total images distributed across four ripeness classes including Over-ripe, Ripe, Under-ripe, and Unripe. Figure 1 presents an example of palm bunch images of each ripeness class. The dataset was partitioned into training, validation, and test splits following a stratified 70/20/10 ratio. Table 1 summarizes the class distribution.

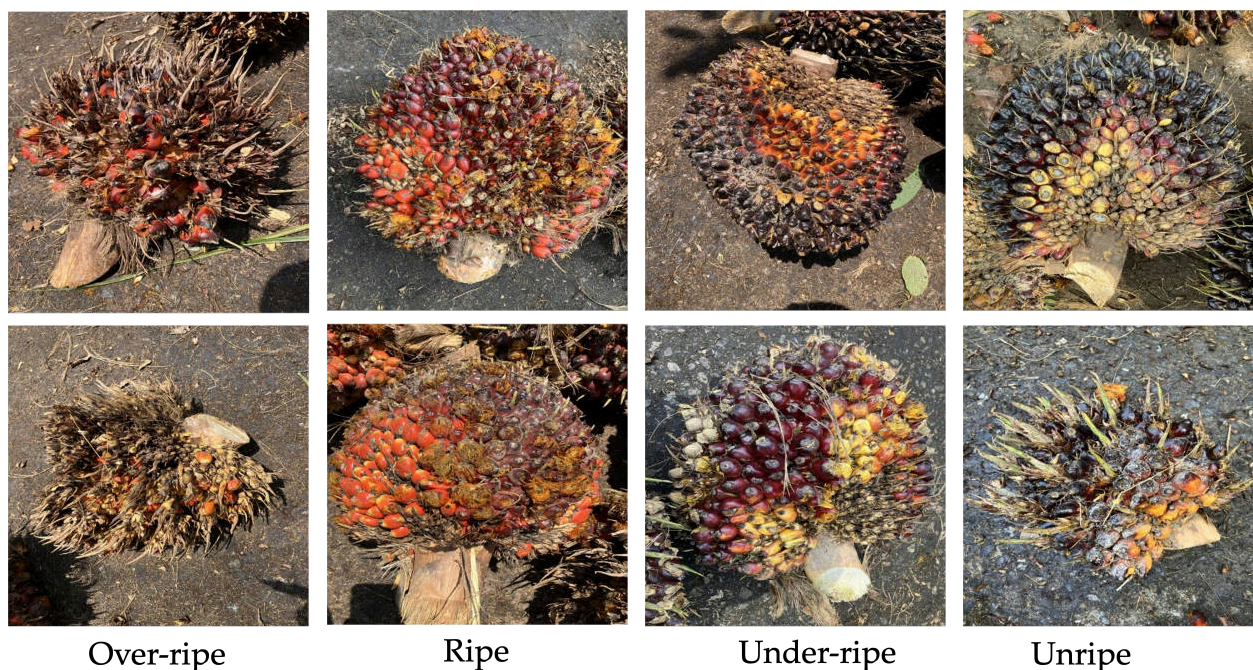


Figure 1. Oil palm fresh fruit bunch images from the dataset collected under real palm ramp conditions.

Table 1. Stratified dataset splitting distribution across classes.

Class Label	Total Samples	Train (70%)	Validation (20%)	Test (10%)
Over-ripe	112	78	22	12
Ripe	231	161	46	24
Under-ripe	334	233	66	35
Unripe	180	126	36	18
Total	857	598	170	89

In real-world environments, the number of images for each ripeness level is not equal. The data was collected from a palm ramp where most of the palm bunch are at the Ripe or Under-ripe stage. The farmers harvest the FFB when the ripeness levels are Ripe or Under-ripe only. Thus, the sample of Over-ripe and Unripe are rarely found in the palm ramp reception. In addition, most local farmers tend to collect the palm bunch in the under-ripe state to prevent the palm bunch from reaching the over-ripe stage due to long harvesting process and transport the bunches to the palm ramp. For this reason, our dataset exhibits a class imbalance which we will address in this proposed work.

2.1.2. Class Imbalance Characterization

To observe the class imbalance in our dataset, we adopt the imbalance ratio (IR), which computes the ratio of the number of minority class relative to the other classes. Table 2 presents the imbalance ratio (IR) relative to the minority class (Over-ripe).

The results show that the imbalance ratio between the majority class (Under-ripe, $n = 233$) and the minority class (Over-ripe, $n = 78$) was 3:1. The IR value also reveals that the Over-ripe sample size is lower compared to Ripe and Unripe classes by factors of $2.1\times$ and $1.6\times$ respectively. In addition, we can observe that the sample counts of Ripe and Unripe classes are approximately $2\times$ lower than the Under-ripe class.

Table 2. Baseline class distribution, Sample share, and Imbalance categories.

Class	Training Sample	Share %	IR	Category
Over-ripe	78	13.0	$1.0\times$ (ref.)	Minority
Ripe	161	26.9	$2.1\times$	Moderate
Under-ripe	233	39.0	$3.0\times$	Majority
Unripe	126	21.1	$1.6\times$	Moderate
Total	598	100.0		

2.2. Data Augmentation Techniques for Class Imbalance

2.2.1. Photometric and Geometric Transformations

Photometric and Geometric transformations (PG) have become standard data augmentation strategies to improve dataset generalization. The photometric augmentation applies color-space modifications to images such as brightness adjustment, contrast modification and color jittering [24]. The geometric transformations are rotation, flipping, scaling, cropping and affine transformations [25].

However, traditional PG has limitations when dealing with severely imbalanced datasets [26]. The transformations are constrained by the manifold of the original data, which can result in a limited diversity of augmented samples [27,28]. For this reason, traditional augmentation paired with more sophisticated generative approaches is presented in this work.

2.2.2. Image Mixing-Based Methods

Several works have adopted cut-paste-based methods to generate training samples for models to learn smoother decision boundaries and improve generalization. CutMix [29] replaces a rectangular region of a training image with the corresponding region from a second image and blends the labels proportionally to the area ratio. Using an enhanced CutMix approach, ref. [30] achieved an average performance increase of 1.5% compared to alternative methods. Mixup [31] method performs pixel-level linear interpolation between two samples with proportional label blending. In a similar approach, [32] integrated a Mixup strategy with diffusion model-generated synthetic images, achieving a 7.9% performance improvement over existing methods.

In our domain, ripeness levels are classified by evaluating the global appearance of the FFB. However, a single FFB often consists of different levels of ripeness. For instance, an FFB may be globally classified as Ripe even though some parts of the bunch are still under-ripe. For this reason, in a fine-grained task such as ripeness grading where adjacent classes (e.g., Ripe and Over-ripe) share similar visual features, cross-class mixing can produce ambiguous composite images that harm inter-class boundary learning.

2.2.3. Boosting Augmentation with YOCO

The YOCO (You Only Cut Once) [33] framework addresses the label-ambiguity issue of CutMix by implementing the augmentation on only one half of the image. YOCO addresses this issue by relocating transformations to the half-image level while maintaining strict adherence to label preservation. The YOCO augmentation technique is illustrated in Equation (1).

$$\tilde{\mathbf{x}} = \begin{cases} \mathcal{A}(\mathbf{x}_L) \parallel \mathbf{x}_R, & \text{if } r \leq 0.5, \\ \mathbf{x}_L \parallel \mathcal{A}(\mathbf{x}_R), & \text{otherwise.} \end{cases} \quad (1)$$

where $r \sim \mathcal{U}(0,1)$, $\tilde{\mathbf{x}}$ denotes the YOCO output, $\mathcal{A}(\cdot)$ denotes an augmentation function, and $\mathbf{x}_L$ and $\mathbf{x}_R$ denote the left (or top) and right (or bottom) halves, respectively.

The algorithmic pipeline of the YOCO framework proceeds through the following sequential operations:

- The halving step: Given an input image $\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$, YOCO selects a random split axis $a \in \{\text{horizontal, vertical}\}$ and then cut down the middle into two equal halves ($\mathbf{x}_L$ and $\mathbf{x}_R$).
- Independent augmentation: The exact same underlying augmentation function $\mathcal{A}$ (e.g., color jitter, random erasing) is called independently on the two halves.
- Concatenation: The halved samples are stitched back together using the concatenation operator ($\parallel$). Because the content originates entirely from the same parent image, the label of $\tilde{\mathbf{x}}$ remains identical to that of $\mathbf{x}$, as YOCO is strictly label-preserving.

2.3. Proposed Method: Intra-Class Half-Mixing with Geometric Augmentation

Palm bunches exhibit varied color across their surface regions. Applying photometric augmentation alone does not capture the structural diversity of real palm bunches. YOCO augmentation produces samples that better reflect the natural surface variation of palm bunches. However, our dataset exhibits a high class imbalance that photometric-geometric augmentation or standard YOCO alone may not adequately address. For this reason, we propose an intra-class half-mixing strategy inspired by YOCO with geometric augmentation to generalize the dataset and address the class imbalance problem. The core novelty of this work is built upon two sequential components: intra-class YOCO mixing and additional geometric transformation. Figure 2 illustrates an overview of the proposed augmentation mechanism.

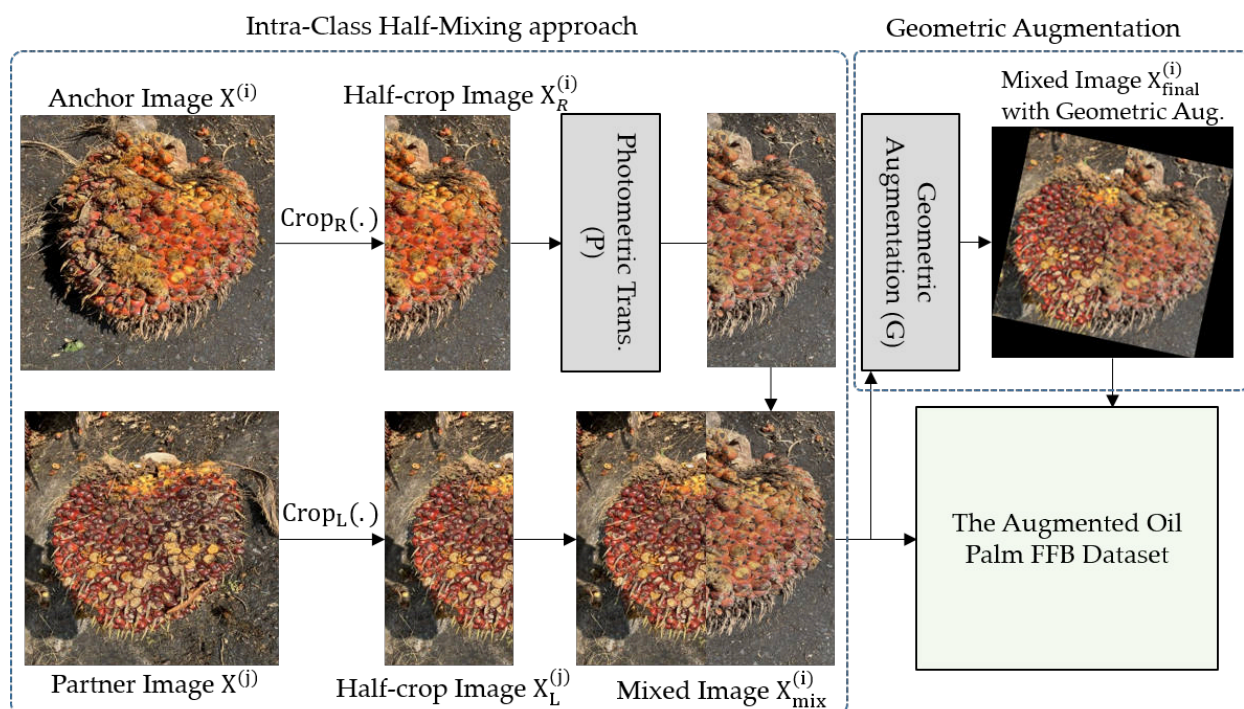


Figure 2. Overview of the proposed augmentation process when the left side of the anchor image is removed and its right side is stitched with the left side of a partner image from the same class.

2.3.1. Component 1: Intra-Class Half-Mixing Strategy

Whereas standard YOCO applies an augmentation function $\mathcal{A}$ to one half of the same image (Equation (1)), our proposed variant introduces an intra-class patch-swapping mechanism. Instead of transforming the pixels of the anchor image, this component

replaces one half of the image with a random crop taken from a different image of the exact same category.

Formally, let $(\mathbf{x}^{(i)}, y^{(i)})$ be the anchor training sample. A mixing partner $\mathbf{x}^{(j)}$ is then randomly selected from all other images belonging to the same class. This is defined as sampling uniformly from the subset $\mathcal{D}_{y^{(i)}} \setminus \{\mathbf{x}^{(i)}\}$ (All training images of the same class excluding the anchor). This proposed mechanism ensures label preservation without requiring mathematical label smoothing methods. The proposed mechanism when applying a vertical cut can be written as Equation (2).

$$\tilde{\mathbf{x}}_{\text{mix}}^{(i)} = \begin{cases} \text{crop}_L(\mathbf{x}^{(j)}) \parallel \mathcal{P}(\mathbf{x}_R^{(i)}), & \text{with probability } q, \\ \mathcal{P}(\mathbf{x}_L^{(i)}) \parallel \text{crop}_R(\mathbf{x}^{(j)}), & \text{with probability } q, \\ \text{YOCO}_{\mathcal{P}}(\mathbf{x}^{(i)}), & \text{with probability } 1 - 2q. \end{cases} \quad (2)$$

where $\mathcal{P}(\cdot)$ denotes the photometric augmentation function, $\tilde{\mathbf{x}}_{\text{mix}}^{(i)}$ denotes the final augmented hybrid image tensor, $\mathbf{x}^{(i)}$ represents the current anchor training image, $\mathbf{x}^{(j)}$ is the intra-class mixing partner image, $\text{crop}_{L/R}(\cdot)$ is the left (or top) or right (or bottom) half-crop of the designated regions, and q denotes the mixing probability hyperparameter where $q \in (0, 0.5)$.

For convenience of notation, the subscripts L and R explicitly denote the left and right halves of the image when a vertical split axis is used, but correspond symmetrically to the Top and Bottom halves, respectively, when a horizontal split axis is applied. The proposed framework evaluates a three-way probabilistic policy for each anchor image $\mathbf{x}^{(i)}$ to balance structural context variation with localized feature transformations. The three possible outputs of the proposed work using vertical split axis are presented as follows:

1. Left-side patch swap case ($\text{crop}_L(\mathbf{x}^{(j)}) \parallel \mathcal{P}(\mathbf{x}_R^{(i)})$): The left side of the anchor image is substituted with an intra-class crop $\text{crop}_L(\mathbf{x}^{(j)})$, while a photometric transformation $\mathcal{P}(\cdot)$ is applied to the remaining right bisection $\mathbf{x}_R^{(i)}$.
2. Right-side patch swap case ($\mathcal{P}(\mathbf{x}_L^{(i)}) \parallel \text{crop}_R(\mathbf{x}^{(j)})$): This case applies photometric distortion to the left half $\mathcal{P}(\mathbf{x}_L^{(i)})$ while swapping the right half with $\text{crop}_R(\mathbf{x}^{(j)})$.
3. Fallback case: This case serves as a baseline single-image policy. It defaults to $\text{YOCO}_{\mathcal{P}}(\mathbf{x}^{(i)})$, which applies the photometric function $\mathcal{P}(\cdot)$ to only one half of the isolated anchor image.

By setting the mixing parameter to $q = 0.25$, the framework allocates a combined 50% probability to intra-class patch-swapping operations (25% for substituting the left region and 25% for the right region). The remaining 50% probability ($1 - 2q$) acts as a fallback mechanism that defaults to standard single-image processing.

2.3.2. Component 2: Geometric Augmentation

The component 1 synthesizes variations to enhance the overall diversity of the training dataset. However, only photometric transformations are applied, which may not accurately depict real-world data that features a variety of viewing angles. In addition, the first step creates an artificial vertical or horizontal boundary line through the middle, and deep neural networks might accidentally learn that harsh line as a feature of the dataset.

For this reason, we adopt geometric augmentation in this stage to force the network to disregard the synthetic seam and focus entirely on the visual content of the two mixed halves. The mathematical formulation of this component is expressed in Equation (3).

$$\tilde{\mathbf{x}}_{\text{final}}^{(i)} = \mathcal{G}(\tilde{\mathbf{x}}_{\text{mix}}^{(i)}; \theta, s) \quad (3)$$

where $\tilde{\mathbf{x}}_{\text{final}}^{(i)}$ denotes the final transformed output image tensor, $\tilde{\mathbf{x}}_{\text{mix}}^{(i)}$ represents the output tensor from the first component, and $\mathcal{G}(\cdot)$ denotes an affine transformation parameterized by a random rotation angle θ and a spatial scaling factor s .

Specifically, the intermediate tensor $\tilde{\mathbf{x}}_{\text{mix}}^{(i)}$ is subsequently passed through this geometric augmentation stage. The transformation parameters are stochastically sampled, where the rotation angle is drawn from a uniform distribution $\theta \sim \mathcal{U}(-20^\circ, +20^\circ)$ and the spatial scaling factor follows $s \sim \mathcal{U}(0.9, 1.3)$.

Functionally, this subsequent geometric warping serves two critical architectural purposes. It exposes the network to variations in object orientation and scale. In addition, this method effectively perturbs the rigid, artificial bisection boundaries introduced during the first component. By rotating and scaling the composite seam, this geometric regularization prevents the deep layers from optimizing for localized stitching artifacts, thus forcing the model to extract more generalized semantic representations. This two-stage design is motivated by the real-world data collection setting, where palm FFB images naturally show variation in color and texture among different bunch instances of the same ripeness stage, and in viewing angle and scale.

2.4. Data Augmentation Results

2.4.1. Augmentation Configuration Summary

To evaluate the efficacy of our proposed work, we conduct multiple experiments utilizing different augmentation techniques. Table 3 provides the detailed specifications for each augmentation configurations evaluated in this work.

Table 3. Augmentation configuration specifications. λ represents the mixing coefficient.

Config.	Transformations and Operational Parameters Applied
E0 (Baseline)	No augmentation applied. Images are downsampled to a fixed spatial resolution of $224 \times 224 \times 3$.
E1 (Standard Aug.)	Geometric: Random horizontal/vertical flipping, random rotation $\theta \sim \mathcal{U}(-15^\circ, +15^\circ)$, and scaling factor $s \sim \mathcal{U}(0.8, 1.2)$. Photometric: Color jittering (brightness/contrast ± 0.2 , saturation ± 0.1 , hue ± 0.05) and Gaussian blurring ($\sigma \in [0.1, 2.0]$ with $p = 0.2$).
E2 (CutMix)	A patch replacement via uniform dataset sampling. A rectangular region of size $W\sqrt{1-\lambda} \times H\sqrt{1-\lambda}$ is cropped from a randomly selected partner image and pasted onto the anchor, with $\lambda \sim \text{Beta}(1.0, 1.0)$. The composite is assigned the label of the source image retaining the larger pixel area.
E3 (YOCO Baseline)	Baseline YOCO framework splitting the image into two spatial halves, applying independent photometric and geometric transformations to one bisection half.
E4 (Intra-class Half-Mixing)	Label-preserving partitioning where the target bisection half is replaced by a half-crop from a randomly selected intra-class partner image (Equation (3)).
E5 (Proposed)	The complete proposed architecture compounding the E4 intra-class mixing framework with a subsequent global geometric transformation stage consisting of random rotation ($\pm 20^\circ$) and scaling.

The baseline configuration (E0) applies no augmentation beyond mandatory spatial resizing to $224 \times 224 \times 3$, serving as the reference point against which all subsequent configurations are measured.

E1 applies standard geometric and photometric transformations independently to each training image. Geometric operations include random horizontal and vertical flipping, rotation $\theta \sim \mathcal{U}(-15^\circ, +15^\circ)$, and scaling $s \sim \mathcal{U}(0.8, 1.2)$. Photometric operations include color jittering (brightness and contrast ± 0.2 , saturation ± 0.1 , hue ± 0.05) and Gaussian

blurring ($\sigma \in [0.1, 2.0]$), applied with probability $p = 0.2$). All operations are applied to the full image without any region-based or cross-image mixing.

E2 applies CutMix through uniform sampling of image pairs. A rectangular patch of size $W\sqrt{1-\lambda} \times H\sqrt{1-\lambda}$, where $\lambda \sim \text{Beta}(1.0, 1.0)$, is cropped from a randomly selected image and pasted onto the anchor image. The resulting composite is assigned the label of the image occupying the larger pixel area. This allows training with a single hard label and standard cross-entropy loss.

E3 applies the baseline YOCO framework [33] per image. The image is divided into two equal spatial halves along a randomly chosen axis, and independent photometric and geometric transformations are applied to one bisection half while the other is left unmodified. Because both halves originate from the same source image, the original label is preserved exactly.

E4 extends the half-splitting procedure of E3 by replacing the target bisection half with a half-crop from a randomly selected partner image belonging to the same ripeness class, following Equation (2). As both halves of the resulting composite belong to the same class, the label is retained without any blending. E4 is therefore strictly label-preserving despite drawing visual content from two distinct images.

E5, the proposed method, applies a global geometric transformation after the E4 intra-class mixing step, as defined in Equation (3). The composite image is randomly rotated by up to $\pm 20^\circ$ and scaled. This additional stage reduces the visibility of the artificial stitching boundary introduced by half-image mixing. Consequently, it prevents the model from learning the stitching boundary as a spurious spatial feature while preserving label consistency.

2.4.2. Augmented FFB Ripeness Dataset

Table 4 reports the per-class sample counts in the training split for each configuration, the total number of training samples, and the maximum imbalance ratio (IR), which is calculated as the ratio of the largest to the smallest class count.

Table 4. Distribution of training samples and maximum Imbalance Ratios (IR) across experimental configurations.

Class/Metric	E0 (Baseline)	E1	E2	E3	E4	E5 (Proposed)
Over-ripe	78	234	156	234	234	468
Ripe	161	483	322	483	322	644
Under-ripe	233	699	466	699	233	466
Unripe	126	378	252	378	252	504
Total	598	1794	1196	1794	1041	2082
Max. IR	$3.0\times$	$3.0\times$	$3.0\times$	$3.0\times$	$1.4\times$	$1.4\times$

The results show that E1, E2, and E3 retain the original baseline imbalance ($3\times$), as these configurations apply augmentation randomly across all classes with fixed multipliers of $3\times$, $2\times$, and $3\times$, respectively. On the other hand, E4 and E5 generate synthetic images that are able to upsample minority classes by leveraging a class-aware augmentation strategy rather than a uniform multiplier.

For E1 and E3, two augmented copies are generated per original image and saved alongside the originals, resulting in a uniform $3\times$ multiplier for every class. For E2, one mixed copy is generated per original image, producing a uniform $2\times$ increase. For E4, the number of mixed copies per original image is determined class-specifically by Equation (4).

$$n_{\text{aug},c} = \left\lceil \frac{n_{\text{max}} - n_c}{n_c} \right\rceil \quad (4)$$

where n_c is the baseline training count of class c and $n_{\max}$ is the count of the majority class (Under-ripe, $n_{\max} = 233$).

The total number of samples for class c after E4 augmentation is calculated as $n_c(1 + n_{\text{aug},c})$. Applying Equation (4) to the baseline class counts in Table 1 yields $n_{\text{aug},c} = 2$ for Over-ripe, 1 for Ripe and Unripe, and 0 for Under-ripe. These values produce class-specific multipliers of $3\times$, $2\times$, $1\times$, and $2\times$ for Over-ripe, Ripe, Under-ripe, and Unripe, respectively. For E5, the geometric augmentation stage copies every image already in the E4 dataset and generates one additional variant per image, exactly doubling the E4 sample count and yielding per-class multipliers of $6\times$, $4\times$, $2\times$, and $4\times$ for Over-ripe, Ripe, Under-ripe, and Unripe respectively. An overview of the training samples produced by our augmentation strategies is illustrated in Figure 3.

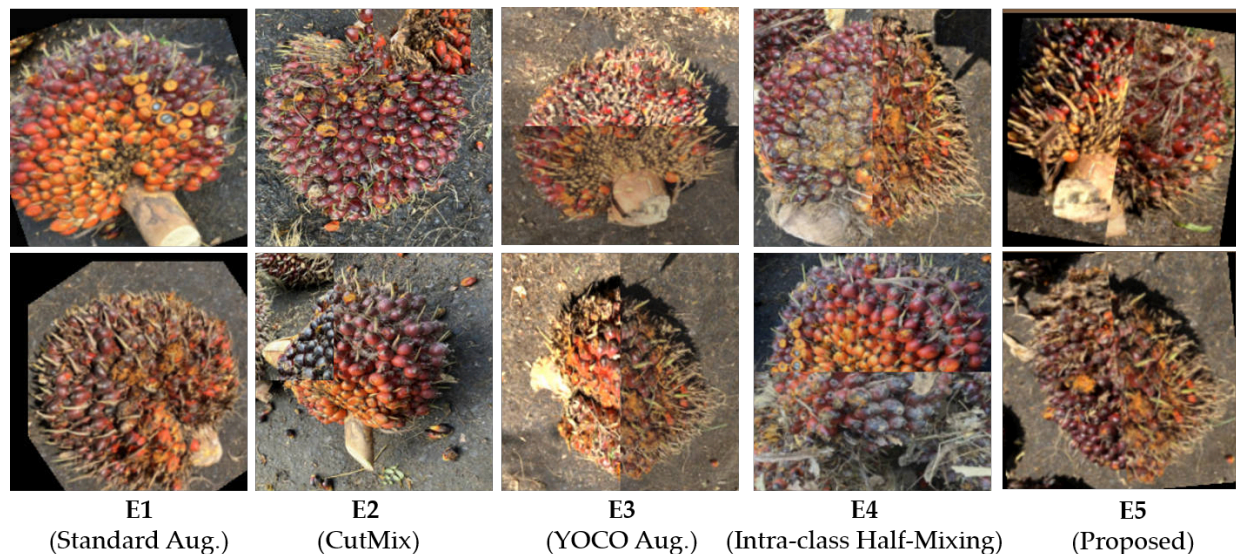


Figure 3. Visual comparison of Ripe class samples across augmentation configuration E0–E5.

Figure 4 illustrates the proposed E5 augmentation applied to two actual Over-ripe (minority class) training images from the collected dataset. The anchor and partner images were captured under different illumination conditions and viewing angles, representing the natural visual variability present within the minority class.

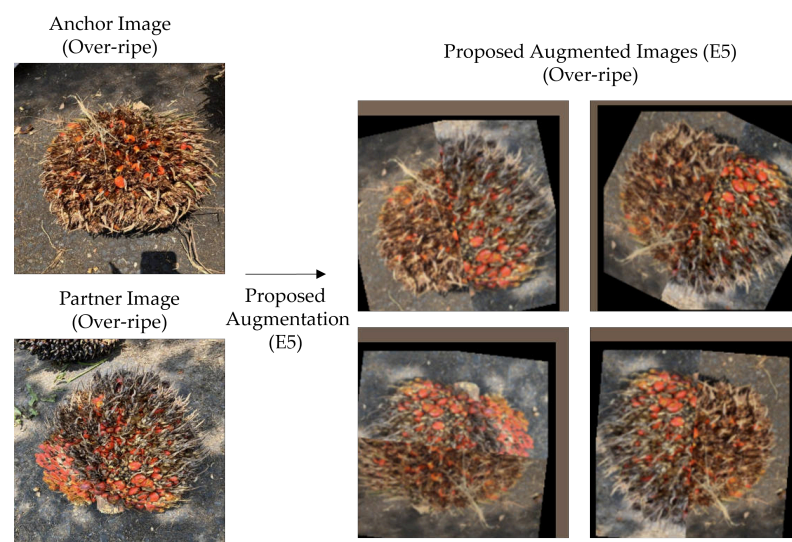


Figure 4. Example of images dataset in the proposed E5 augmentation for the Over-ripe minority class. (Left): anchor and partner images (same class, different instances). (Right): four E5-generated composites retaining the Over-ripe label.

The four generated composites demonstrate how E5 combines spatial regions from two distinct Over-ripe instances through intra-class half-mixing, then applies rotation and zoom-in to disrupt the stitching boundary. The dark corners in the augmented images are the affine padding introduced by the rotation stage, which prevents the model from anchoring its minority-class representation to a fixed spatial boundary. Because both source images belong to the same ripeness class, the Over-ripe label is preserved exactly in all four composites.

2.5. *Lightweight Deep Learning-Based Palm Ripeness Classification*

Our objective is to implement a palm ripeness classification framework capable of executing efficiently on computationally constrained edge devices. To achieve this, we focus on lightweight deep neural network architectures that provide a promising classification performance with optimal resource utilization efficiency. The lightweight deep learning approach can be categorized into two main types: conventional classification networks and object detection networks. In this study, we explore and evaluate several architectures to perform the ripeness classification task. The lightweight architectures included in our experiments consist of MobileNetV2, EfficientNet-based models, and YOLO-based models.

The six backbone architectures evaluated in this study were selected according to three criteria. First, each architecture must be classified as lightweight, with a parameter count suitable for deployment on resource-constrained edge hardware typical of mill reception stations (target: under 10 M parameters). Second, each selected architecture had previously been applied to fruit ripeness or FFB classification in the literature, providing an empirical basis for its expected baseline performance in this domain. Third, the selected models represent two architectural families including conventional CNN classifiers and detection-derived classification models. This allows the proposed augmentation strategy to be evaluated across different feature-extraction approaches rather than being tailored to a single model family.

2.5.1. MobileNetV2-Based Ripeness Classification

MobileNetV2 [34] introduces inverted residual blocks with linear bottlenecks and depthwise separable convolutions, resulting in a compact parameter count and suitable for resource-constrained mobile and edge-device deployment. Several studies have shown the efficacy of this architecture on palm fruit ripeness datasets. For instance, prior work deployed MobileNet on smartphones for real-world, on-device palm fruit ripeness classification, achieving an accuracy of 85% [35]. Similarly, an alternative study evaluated multiple MobileNet variants on a 6-class palm fresh fruit bunch (FFB) dataset and was able to achieve a peak classification accuracy of 81% [10].

2.5.2. EfficientNetV2 Series

EfficientNetV2 [36], developed through neural architecture search (NAS), systematically scales network depth, width, and resolution to maximize computational efficiency. In a previous work, an optimized EfficientNet variant was employed for FFB classification in [37]. The proposed work achieved 97.16% accuracy on a 5-class FFB dataset. Furthermore, a related study [38] deployed the EfficientDet-Lite CNN architecture (an extended version of EfficientNet for object detection tasks) for oil palm ripeness level classification and yielded an accuracy of 84.0%. Based on these foundations, we evaluate two EfficientNetV2-based architectures in this study:

- **EfficientNetV2-B0:** This configuration represents the baseline variant (leveraging the standard default architecture) with a parameter count of ≈ 6 M.

- **EfficientNetV2-B1**: This configuration represents an architecturally scaled variant (leveraging both wider and deeper layers) with a parameter count of ≈ 7 M.

2.5.3. YOLO-Based Classification

YOLO (You Only Look Once) [14] architectures have become increasingly popular for palm fruit ripeness monitoring due to their real-time inference speed and ability to simultaneously detect and classify multiple objects in a single forward pass. For instance, a real-time grading system using a modified YOLOv4 backbone on smartphones [15] and another YOLOv4-based system [39] demonstrated the suitability of the model for on-device field deployment. Additionally, YOLOv8n-Segmentation model (the nano variant with segmentation capabilities) was previously employed for preprocessing in [11], achieving 98.07% accuracy when combined with ShuffleNetV2 [40].

In this work, we evaluate three nano-scale YOLO classification variants, which are characterized as follows:

- **YOLOv8n-cls** [41]: This model employs C2f (Cross Stage Partial with two convolutions) modules throughout its backbone and neck features, while maintaining a parameter count of ≈ 1.4 M.
- **YOLO11n-cls** [42]: This architecture introduces refined multi-stage feature extraction blocks and advanced attention mechanisms, utilizing a parameter count of ≈ 1.5 M.
- **YOLO26n-cls** [43]: This is the latest version among the YOLO variants. It implements a deeper feature aggregation neck and a dedicated classification head relative to YOLOv8n, totaling ≈ 1.5 M parameters.

2.5.4. Model Size and Inference Latency

To compare the computational characteristics of the evaluated lightweight architectures under a common hardware platform, Table 5 reports the exact parameter count, model size, and single-image inference latency for all six evaluated architectures. Inference latency was measured as the mean and standard deviation over 200 single-image forward passes on an NVIDIA RTX 3080 GPU (12 GB VRAM). Disk size for CNN models reflects the full training checkpoint, including optimizer state. The FP32/FP16 columns report the theoretical weight-only footprint. The reported latency values are intended for relative comparison under identical hardware conditions and should not be interpreted as the expected inference latency on embedded deployment platforms.

The results show that the three YOLO-family models are substantially more compact than the CNN-based models across all metrics. YOLOv8n-cls, YOLO11n-cls, and YOLO26n-cls each occupy under 6 MB at FP32 precision and under 3 MB at FP16, with deployment-ready sizes of approximately 2.83–3.05 MB. Their single-image inference latency ranges from 3.59 ± 0.82 ms to 4.64 ± 0.94 ms on the evaluated hardware platform. The CNN-based models range from 10.02 MB (MobileNetV2, FP32) to 27.84 MB (EfficientNetV2B1, FP32), remaining well within the memory budget of many embedded inference platforms when quantized to FP16 (5.01–13.92 MB).

Table 5. Model size and single-image inference latency for all evaluated architectures, measured on an NVIDIA RTX 3080 GPU (12 GB VRAM).

Model	Params (M)	Disk (MB)	FP32 (MB)	FP16 (MB)	Latency (ms)
MobileNetV2	2.63	29.98	10.02	5.01	43.42 ± 7.84
EfficientNetV2B0	6.29	71.76	23.98	11.99	49.67 ± 13.43
EfficientNetV2B1	7.30	83.33	27.84	13.92	49.47 ± 5.32
YOLOv8n-cls	1.44	2.83	5.51	2.75	3.59 ± 0.82
YOLO11n-cls	1.54	3.05	5.86	2.93	4.41 ± 1.00
YOLO26n-cls	1.54	3.05	5.86	2.93	4.64 ± 0.94

These results indicate that all six architectures have relatively small memory footprints and favorable computational characteristics. However, inference latency was measured only on a desktop-class NVIDIA RTX 3080 GPU and therefore does not directly represent the runtime performance that would be achieved on embedded devices with more limited computational resources. Consequently, the reported latency values should be interpreted as a reproducible benchmark for relative model comparison rather than as deployment performance.

2.6. Model Development and Training

The training of the classification models was performed on a desktop computer equipped with an NVIDIA RTX 3080 GPU (12 GB VRAM) using cuDNN version 12.4 and implemented in Python version 3.10.12. The hyperparameter configuration and optimization policies selected for training the classification models are described in Table 6.

Table 6. Hyperparameter configuration and optimization policy for model training.

Parameter	Value
Optimization method	AdamW
Initial learning rate	0.001
Batch size	16
Total epochs	100
Input resolution	$224 \times 224 \times 3$
Loss function	Class-weighted cross-entropy

The hyperparameter configuration reported in Table 6 was selected based on commonly used settings for transfer-learning-based image classification and preliminary validation experiments. A systematic hyperparameter search, such as grid search, random search, or Bayesian optimization, was not performed because the primary objective of this study was to compare augmentation strategies under a fixed and consistent training protocol rather than to optimize each model–augmentation combination individually.

For each architecture, the same training configuration was used across all six augmentation settings. Therefore, the within-model performance differences primarily reflect the effect of the augmentation strategy and the resulting training-data distribution. The CNN-based models were trained for 100 epochs using the AdamW optimizer [44], an initial learning rate of 0.001, and a batch size of 16. The dropout rate and L2 regularization strength used in the classification head were selected from commonly adopted settings for small-to-medium transfer-learning datasets and were not tuned separately for individual models or augmentation configurations. For the YOLO-based classification models, training was conducted for 100 epochs with a batch size of 16 using AdamW optimizer as the CNN-based models. Thus, the optimizer-selection procedure remained consistent across the six augmentation configurations.

To further mitigate class imbalance, a normalized inverse-frequency class weighting scheme was applied during the training of the EfficientNet and MobileNet models across all six augmentation configurations. For each configuration, class weights were computed directly from its corresponding training set using Equation (5). The denominator normalizes the raw inverse-frequency weights by their mean, so that the resulting weights average to 1.0 across classes.

$$w_c = \frac{N / (C \cdot n_c)}{\frac{1}{C} \sum_{k=1}^C N / (C \cdot n_k)}, \quad c = 1, \dots, C \quad (5)$$

where N denotes the total training sample count for the given configuration, C represents the number of classes, and n_c is the sample count of class c within that configuration.

Table 7 reports the exact normalized weight values for every configuration, computed from the per-class training sample counts in Table 4.

Table 7. Normalized class weights w_c applied during training models for each augmentation configuration.

Class	E0/E1/E3	E2	E4	E5 (Proposed)
Over-ripe	1.6405	1.6405	1.0930	1.0930
Ripe	0.7948	0.7948	0.7943	0.7943
Under-ripe	0.5492	0.5492	1.0977	1.0977
Unripe	1.0155	1.0155	1.0150	1.0150

E0, E1, E2, and E3 produce identical normalized class weights despite differences in total sample counts. Specifically, E1 and E3 apply a uniform $3\times$ increase to all classes, while E2 applies a uniform $2\times$ increase. In contrast, E4 and E5 use class-aware augmentation to rebalance the dataset directly, reducing the maximum imbalance ratio from $3.0\times$ to $1.4\times$, as shown in Table 4. As a result, only a small residual correction is required with normalized class weights ranging from 0.794 to 1.098, compared with 0.549 to 1.641 for E0–E3. These results indicate that the proposed augmentation strategy performs the majority of the imbalance correction at the data level, while class-weighted loss provides only a second correction across all configurations. Therefore, performance differences among these configurations mainly reflect the augmentation strategies and resulting training-set sizes rather than changes in loss weighting.

2.7. Evaluation Metrics

To assess the performance of the deep learning frameworks, we implement a combination of global and class-specific metrics. We adopt global accuracy to evaluate the overall multi-class classification performance across the entire dataset. To provide a more granular evaluation, we further employ metrics derived from the components of the multi-class confusion matrix. Specifically, for each distinct class $c \in \{1, 2, \dots, C\}$, performance is quantitatively mapped using the per-class precision (P_c), recall (R_c), and their harmonic mean, denoted as the per-class F_1 -score ($F_{1,c}$).

The precision metric (P_c) is used to measure the proportion of correctly predicted positive instances out of all predicted positives for class c , formulated as Equation (6).

$$P_c = \frac{TP_c}{TP_c + FP_c} \quad (6)$$

In addition, the recall metric (R_c) evaluates the ability of the model to identify all actual positive instances within class c , and is expressed as Equation (7).

$$R_c = \frac{TP_c}{TP_c + FN_c} \quad (7)$$

where TP_c , FP_c , and FN_c denote the True Positives, False Positives, and False Negatives for a given class c , respectively.

To evaluate model performance when the dataset has an uneven class distribution, we adopt the per-class F_1 -score to provide a balanced metric that accounts for both false positives and false negatives simultaneously. The F_1 -score metric is presented in Equation (8).

$$F_{1,c} = 2 \times \frac{P_c \times R_c}{P_c + R_c} \quad (8)$$

Moreover, we used the macro-averaged F_1 -score presented in Equation (9) to evaluate model generalization on the imbalanced dataset:

$$F_{1,\text{macro}} = \frac{1}{C} \sum_{c=1}^C F_{1,c} \quad (9)$$

where C is the number of classes and $F_{1,c}$ is the F_1 -score of class c .

3. Results

3.1. Overall Accuracy Results

Six augmentation configurations were evaluated under a fixed training protocol. These included the E0 baseline without augmentation, E1 with standard geometric and photometric transformations, E2 with CutMix, E3 with baseline YOCO, and E4 with intra-class half-mixing YOCO. The proposed E5 configuration combines intra-class YOCO mixing with geometric transformations to form a hybrid augmentation framework. Overall accuracy results are presented in Table 8, while macro-averaged F_1 -scores are reported in Table 9. No formal statistical significance testing was conducted; therefore, the reported differences should be interpreted as observed empirical trends rather than statistically confirmed superiority among augmentation strategies.

Table 8. Overall accuracy (%) performance across different augmentation configurations. **Bold** = highest result per model.

Model	E0	E1	E2	E3	E4	E5 (Proposed)
MobileNetV2	88.0	89.6	92.0	90.8	89.6	89.6
EfficientNetV2B0	90.8	94.8	91.2	94.4	90.4	94.8
EfficientNetV2B1	91.6	93.6	91.6	92.4	93.2	94.8
YOLOv8n-cla	83.2	87.6	89.2	90.8	86.8	95.6
YOLO11n-cla	88.8	91.2	84.4	89.6	88.8	92.4
YOLO26n-cla	89.2	92.0	89.6	92.4	88.8	92.8

The proposed E5 configuration achieved the highest observed accuracy for five of the six evaluated models. For the EfficientNet family, E5 increased accuracy relative to the non-augmented baseline by 4.0 percentage points (pp) for EfficientNetV2B0 (90.8% to 94.8%) and by 3.2 pp for EfficientNetV2B1 (91.6% to 94.8%). MobileNetV2 was the only exception, with its highest observed accuracy of 92.0% obtained under E2 (CutMix). Nevertheless, E5 still produced a 1.6 pp improvement over the MobileNetV2 baseline (88.0% to 89.6%).

For the YOLO-family architectures, all three models achieved their highest observed accuracy under the E5 configuration. YOLO11n-cla and YOLO26n-cla each improved by 3.6 pp relative to their respective baselines. The largest observed gain was obtained by YOLOv8n-cla, which improved by 12.4 pp from 83.2% to 95.6%. This was also the highest individual accuracy recorded in the present experiments.

The macro- F_1 results reported in Table 9 show a similar empirical trend to the overall accuracy results. The proposed E5 configuration achieved the highest observed macro- F_1 score for five of the six evaluated models. Notably, YOLOv8n-cla reached a macro- F_1 score of 0.955, representing the highest observed result across all tested models and configurations. For the EfficientNet family, E5 achieved the highest observed macro- F_1 score for both models, reaching 0.946 for EfficientNetV2B0 and 0.948 for EfficientNetV2B1. These results correspond to numerical gains of 0.040 and 0.032 over their respective baselines. Conversely, MobileNetV2 again achieved its highest macro- F_1 score under the E2 configuration (0.927), whereas E5 produced a macro- F_1 score of 0.903. Across the YOLO-family variants,

the proposed E5 framework produced the largest observed macro- F_1 improvement over the baseline, particularly for YOLOv8n-cls, which increased from 0.847 to 0.955.

Table 9. Macro-averaged F_1 -score by model and augmentation configuration. **Bold** = highest result per model.

Model	E0	E1	E2	E3	E4	E5 (Proposed)
MobileNetV2	0.882	0.906	0.927	0.918	0.903	0.903
EfficientNetV2B0	0.906	0.939	0.913	0.941	0.904	0.946
EfficientNetV2B1	0.916	0.937	0.920	0.929	0.937	0.948
YOLOv8n-cls	0.847	0.880	0.896	0.917	0.879	0.955
YOLO11n-cls	0.894	0.920	0.862	0.901	0.899	0.930
YOLO26n-cls	0.899	0.921	0.900	0.923	0.893	0.934

3.2. Per-Class Performance

To evaluate the effectiveness of the proposed framework in addressing class imbalance, we conduct an analysis of the per-class precision (P), recall (R), and F_1 -score for each model across all evaluated configurations.

3.2.1. MobileNetV2

Table 10 presents the per-class performance of MobileNetV2 across all evaluated configurations.

Table 10. Per-class performance metrics for MobileNetV2 across augmentation configurations. **Bold** = highest result per metric per class.

Config	Over-Ripe			Ripe			Under-Ripe			Unripe		
	P	R	F_1	P	R	F_1	P	R	F_1	P	R	F_1
E0	0.913	0.875	0.894	0.900	0.900	0.900	0.896	0.851	0.873	0.820	0.909	0.862
E1	0.923	1.000	0.960	0.942	0.929	0.935	0.889	0.871	0.880	0.839	0.855	0.847
E2	0.923	1.000	0.960	0.904	0.943	0.923	0.928	0.891	0.909	0.926	0.909	0.917
E3	0.923	1.000	0.960	0.880	0.943	0.910	0.944	0.842	0.890	0.881	0.945	0.912
E4	0.920	0.958	0.939	0.903	0.929	0.915	0.890	0.881	0.886	0.887	0.855	0.870
E5 (Proposed)	0.920	0.958	0.939	0.915	0.929	0.922	0.906	0.861	0.883	0.845	0.891	0.867

The MobileNetV2 results indicate that the proposed E5 configuration did not achieve the highest observed overall performance for this architecture. Instead, the E2 (CutMix) configuration achieved the highest observed performance, with perfect Over-ripe recall ($R = 1.000$, $F_1 = 0.960$), together with the highest Under-ripe F_1 -score (0.909) and Unripe F_1 -score (0.917). Despite these results, the proposed framework still delivered an improvement for the minority class compared to the baseline, where the Over-ripe F_1 -score increased from 0.894 (E0) to 0.939 (E5).

3.2.2. EfficientNetV2B0

Table 11 presents the per-class performance of EfficientNetV2B0 across all evaluated configurations. For the Over-ripe minority class, E5 achieved the highest observed precision ($P = 0.920$) and F_1 -score ($F_1 = 0.939$). While E3 (YOCO) achieved the highest observed recall ($R = 1.000$) for this class, its precision was 0.857, resulting in an F_1 -score of 0.923. In comparison, E5 obtained a recall of 0.958 and a higher precision of 0.920, resulting in the highest observed Over-ripe F_1 -score for EfficientNetV2B0. Although E4 achieved an Over-ripe recall score of 0.958, its precision was 0.852 ($F_1 = 0.902$).

Table 11. Per-class performance metrics for EfficientNetV2B0 across augmentation configurations. **Bold** = highest result per metric per class.

Config	Over-Ripe			Ripe			Under-Ripe			Unripe		
	P	R	F ₁	P	R	F ₁	P	R	F ₁	P	R	F ₁
E0	0.852	0.958	0.902	0.952	0.857	0.902	0.874	0.960	0.915	0.959	0.855	0.904
E1	0.852	0.958	0.902	0.984	0.900	0.940	0.935	0.990	0.962	0.981	0.927	0.953
E2	0.885	0.958	0.920	0.939	0.886	0.912	0.911	0.911	0.911	0.895	0.927	0.911
E3	0.857	1.000	0.923	0.954	0.886	0.919	0.942	0.960	0.951	0.981	0.964	0.972
E4	0.852	0.958	0.902	0.922	0.843	0.881	0.902	0.911	0.906	0.912	0.945	0.929
E5 (Proposed)	0.920	0.958	0.939	0.957	0.943	0.950	0.942	0.960	0.951	0.962	0.927	0.944

Across the four ripeness classes, E5 achieved F_1 -scores of 0.939, 0.950, 0.951, and 0.944 for Over-ripe, Ripe, Under-ripe, and Unripe, respectively. Under the fixed experimental protocol, E5 was the only configuration for EfficientNetV2B0 with observed F_1 -scores of at least 0.939 across all four classes.

3.2.3. EfficientNetV2B1

Table 12 outlines the per-class performance metrics for EfficientNetV2B1 across all evaluated augmentation configurations. For the EfficientNetV2B1 architecture, the E3 and E4 configurations obtained the highest observed Over-ripe F_1 -score of 0.960 with perfect recall ($R = 1.000$), while the proposed E5 framework achieved an Over-ripe F_1 -score of 0.941.

Table 12. Per-class performance metrics for EfficientNetV2B1 across augmentation configurations. **Bold** = highest result per metric per class.

Config	Over-Ripe			Ripe			Under-Ripe			Unripe		
	P	R	F ₁	P	R	F ₁	P	R	F ₁	P	R	F ₁
E0	0.885	0.958	0.920	0.929	0.929	0.929	0.895	0.931	0.913	0.959	0.855	0.904
E1	0.920	0.958	0.939	0.955	0.900	0.926	0.931	0.941	0.936	0.930	0.964	0.946
E2	0.920	0.958	0.939	0.914	0.914	0.914	0.903	0.921	0.912	0.942	0.891	0.916
E3	0.923	1.000	0.960	0.929	0.929	0.929	0.929	0.911	0.920	0.909	0.909	0.909
E4	0.923	1.000	0.960	0.984	0.886	0.932	0.906	0.950	0.928	0.927	0.927	0.927
E5 (Proposed)	0.889	1.000	0.941	0.942	0.929	0.935	0.960	0.941	0.950	0.964	0.964	0.964

The proposed E5 configuration obtained an Over-ripe recall of 1.000, an Under-ripe F_1 -score of 0.950, and an Unripe F_1 -score of 0.964. These scores represent the highest value achieved across all configurations and categories for this model. Under the evaluated experimental protocol, E5 was the only configuration that achieved an F_1 -score ≥ 0.935 across all four classes.

3.2.4. YOLOv8n-cl

Table 13 presents the per-class classification results for YOLOv8n-cl under each evaluated configuration. The YOLOv8n-cl architecture under the proposed E5 configuration achieved the highest observed overall result of the present study, with perfect Over-ripe recall ($R = 1.000$, $F_1 = 0.960$) and consistent per-class scores across all classes. This model delivered an F_1 -score for the Ripe, Under-ripe, and Unripe classes of 0.971, 0.956, and 0.935, respectively.

Table 13. Per-class performance metrics for YOLOv8n-cls across augmentation configurations. **Bold** = highest result per metric per class.

Config	Over-Ripe			Ripe			Under-Ripe			Unripe		
	P	R	F ₁	P	R	F ₁	P	R	F ₁	P	R	F ₁
E0	0.917	0.917	0.917	0.919	0.814	0.864	0.833	0.792	0.812	0.721	0.891	0.797
E1	0.913	0.875	0.894	0.908	0.843	0.874	0.887	0.851	0.869	0.815	0.964	0.883
E2	0.857	1.000	0.923	0.922	0.843	0.881	0.876	0.911	0.893	0.906	0.873	0.889
E3	0.923	1.000	0.960	0.903	0.929	0.915	0.908	0.881	0.894	0.907	0.891	0.899
E4	0.923	1.000	0.960	0.909	0.857	0.882	0.816	0.921	0.865	0.909	0.727	0.808
E5 (Proposed)	0.923	1.000	0.960	0.985	0.957	0.971	0.942	0.970	0.956	0.962	0.909	0.935

On the other hand, E4 produced the lowest Unripe F_1 -score (0.808) among the evaluated configurations for this model. The proposed E5 framework showed a higher Unripe F_1 -score than E4, increasing from 0.808 to 0.935. In terms of F_1 -score performance, E5 showed numerical improvements of 0.043 in Over-ripe, 0.107 in Ripe, 0.144 in Under-ripe, and 0.138 in Unripe compared with the baseline setup (E0).

3.2.5. YOLO11n-cls

Table 14 details the per-class precision, recall, and F_1 -scores for the YOLO11n-cls architecture across all six experimental configurations. For the YOLO11n-cls model, the E1, E4, and E5 configurations all obtained perfect Over-ripe recall ($R = 1.000$) and the highest observed Over-ripe F_1 -score ($F_1 = 0.960$).

Table 14. Per-class performance metrics for YOLO11n-cls across augmentation configurations. **Bold** = highest result per metric per class.

Config	Over-Ripe			Ripe			Under-Ripe			Unripe		
	P	R	F ₁	P	R	F ₁	P	R	F ₁	P	R	F ₁
E0	0.917	0.917	0.917	0.909	0.857	0.882	0.881	0.881	0.881	0.864	0.927	0.895
E1	0.923	1.000	0.960	0.969	0.900	0.933	0.909	0.891	0.900	0.850	0.927	0.887
E2	0.920	0.958	0.939	0.915	0.771	0.837	0.828	0.812	0.820	0.776	0.945	0.852
E3	0.920	0.958	0.939	0.913	0.900	0.906	0.900	0.891	0.896	0.857	0.873	0.865
E4	0.923	1.000	0.960	0.912	0.886	0.899	0.889	0.871	0.880	0.842	0.873	0.857
E5 (Proposed)	0.923	1.000	0.960	0.984	0.900	0.940	0.912	0.921	0.916	0.879	0.927	0.903

The proposed E5 framework achieved perfect minority-class recall while obtaining the highest observed F_1 -scores for the remaining categories, yielding a Ripe F_1 -score of 0.940, an Under-ripe F_1 -score of 0.916, and an Unripe F_1 -score of 0.903. Under the evaluated experimental protocol, E5 was the only configuration that maintained an F_1 -score ≥ 0.90 across all four classes.

3.2.6. YOLO26n-cls

Table 15 presents the class-specific performance breakdown for the deeper YOLO26n-cls model under each evaluated augmentation setup. The results show that the E3 configuration achieved the highest observed performance on the Ripe and Under-ripe classes, with F_1 -scores of 0.935 and 0.925, respectively. In contrast, the E4 configuration showed lower F_1 -scores across all classes compared with the best observed results for this model.

Table 15. Per-class performance metrics for YOLO26n-cl5 across augmentation configurations. **Bold** = highest result per metric per class.

Config	Over-Ripe			Ripe			Under-Ripe			Unripe		
	P	R	F ₁	P	R	F ₁	P	R	F ₁	P	R	F ₁
E0	0.920	0.958	0.939	0.912	0.886	0.899	0.853	0.921	0.886	0.938	0.818	0.874
E1	0.920	0.958	0.939	0.955	0.914	0.934	0.882	0.960	0.919	0.958	0.836	0.893
E2	0.885	0.958	0.920	0.897	0.871	0.884	0.885	0.911	0.898	0.923	0.873	0.897
E3	0.857	1.000	0.923	0.942	0.929	0.935	0.939	0.911	0.925	0.909	0.909	0.909
E4	0.917	0.917	0.917	0.886	0.886	0.886	0.860	0.911	0.885	0.939	0.836	0.885
E5 (Proposed)	0.923	1.000	0.960	0.929	0.929	0.929	0.921	0.921	0.921	0.943	0.909	0.926

The proposed E5 configuration achieved perfect recall and the highest observed precision for the Over-ripe minority class, resulting in an F_1 -score of 0.960. E5 also produced the highest Unripe F_1 -score (0.926), and its Ripe F_1 -score (0.929) and Under-ripe F_1 -score (0.921) remained close to the E3 configuration.

3.3. Error Analysis

Figure 5 presents the row-normalized confusion matrices for all six models under E5. Each cell represents the proportion of samples from a true class assigned to each predicted class. Therefore, the diagonal values correspond to the per-class recall reported in Tables 10–15.

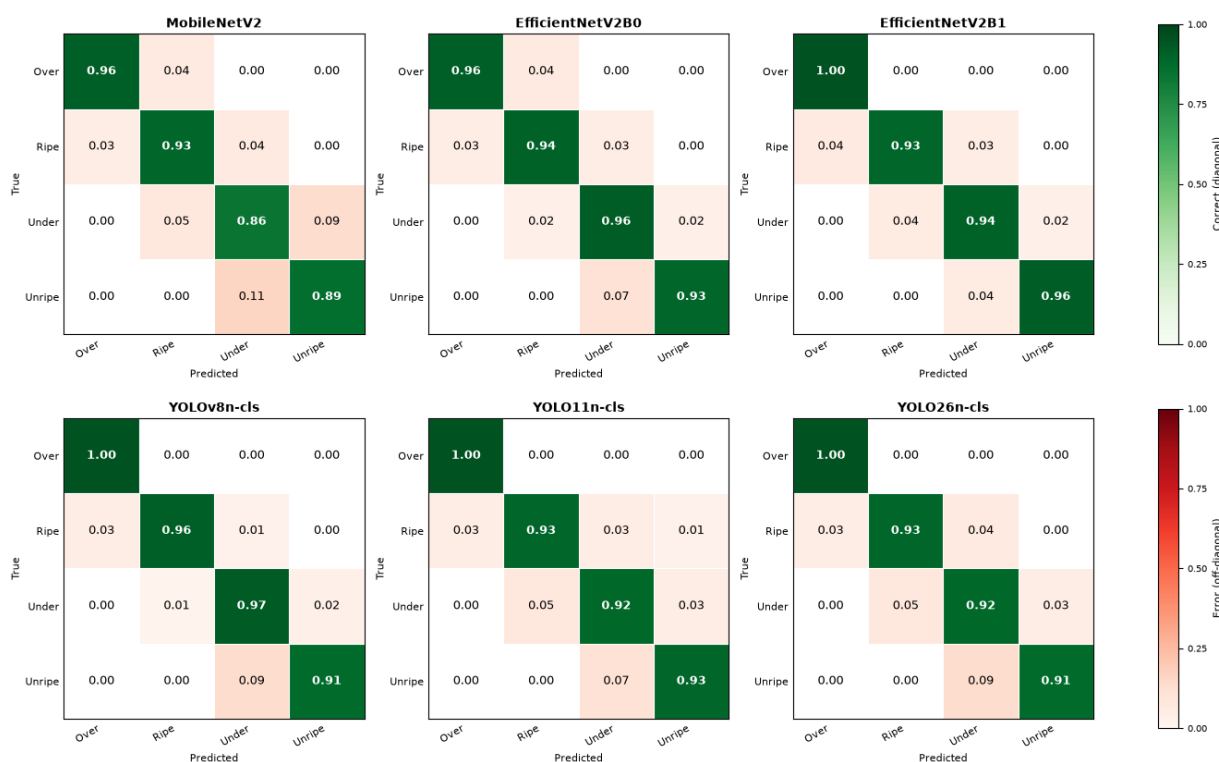


Figure 5. Row-normalized confusion matrices for all six models under the proposed augmentation E5 configuration. Classes: Over = Over-ripe, Ripe = Ripe, Under = Under-ripe, Unripe = Unripe.

Three consistent observations can be drawn from the six confusion matrices. First, all three YOLO-family models achieved perfect Over-ripe recall ($R = 1.000$, diagonal value 1.00), while MobileNetV2 and EfficientNetV2B0 reached $R = 0.958$ (diagonal 0.96) and EfficientNetV2B1 achieved $R = 1.000$. Second, off-diagonal errors were most frequently observed between visually adjacent ripeness stages, particularly between Ripe and Over-

ripe and between Under-ripe and Unripe. Third, MobileNetV2 exhibited the highest off-diagonal values overall, with an Under-ripe to Unripe error rate of 0.09 and an Unripe to Under-ripe error rate of 0.11, representing the largest inter-class confusion observed in this study. Among the YOLO-family models, YOLOv8n-cls exhibited the most balanced per-class recall, with values of 1.00 for Over-ripe, 0.96 for Ripe, 0.97 for Under-ripe, and 0.91 for Unripe. All off-diagonal error rates were 0.03 or lower. By comparison, YOLO11n-cls and YOLO26n-cls demonstrated similar recall patterns, with their primary off-diagonal errors occurring between Under-ripe and Ripe and between Unripe and Under-ripe.

4. Discussion

4.1. Effectiveness of the Proposed Hybrid Augmentation

The experimental results suggest that combining intra-class mixing with geometric augmentation, as implemented in configuration E5, is an effective strategy for addressing class imbalance in oil palm FFB ripeness classification. Five out of six models achieved their highest observed accuracy and macro- F_1 -score under the proposed E5 configuration. Furthermore, the best single result, achieved by YOLOv8n-cls with an accuracy of 95.6% and a macro- F_1 -score of 0.955, represents the highest observed performance across all models and configurations evaluated in this study. In addition, the proposed framework improved minority-class performance while maintaining competitive performance on the majority classes. Under the evaluated experimental protocol, it was the only configuration that consistently achieved an F_1 -score ≥ 0.90 for all four ripeness classes across the YOLO-family models. These observations suggest that the proposed method improves minority-class performance without an evident precision–recall trade-off at the system level.

The intra-class YOCO mixing stage addresses class imbalance by synthesizing spatially diverse composites from different Over-ripe samples. This method increases the effective training diversity for the minority class without introducing cross-class label ambiguity. However, the experimental results indicate that intra-class mixing alone may be insufficient. In the YOLO26n-cls results, the Over-ripe F_1 -score decreased to 0.917. This observation suggests that the stitching boundary in E4 composites may act as a spurious spatial feature that the model partially relies on instead of learning genuine ripeness characteristics.

The geometric stage incorporated in E5 may help alleviate this issue by rotating and rescaling the composite, thereby reducing the likelihood that the model relies on a fixed stitching seam and encouraging greater reliance on morphological and color features. This interpretation is supported by the EfficientNetV2B0 results, where E5 improved Over-ripe precision from 0.857 (E3) to 0.920 while maintaining recall at 0.958. Similarly, for EfficientNetV2B1, E5 achieved perfect Over-ripe recall together with the highest observed majority-class F_1 -scores among the evaluated configurations.

4.2. Over-Ripe Minority Class: Precision–Recall Tradeoff

Across all YOLO-family models, configuration E5 achieved perfect Over-ripe recall ($R = 1.000$) together with the highest observed Over-ripe precision ($P \geq 0.923$), resulting in an F_1 -score of 0.960 for all three architectures. This precision–recall combination is desirable for oil palm FFB ripeness classification because high recall reduces the likelihood of missing Over-ripe bunches, while high precision reduces the likelihood of incorrectly classifying acceptable bunches as Over-ripe. Prior configurations that achieved high recall (E3 and E4) did so with lower precision. Under the evaluated experimental protocol, E5 was the only configuration that achieved both perfect Over-ripe recall and the highest observed Over-ripe precision across all three YOLO architectures.

For the EfficientNet models, the proposed configuration (E5) achieved a different but complementary balance compared with the YOLO variants. It maintained perfect

Over-ripe recall for EfficientNetV2B1 while substantially improving the majority-class F_1 -score (Unripe F_1 : 0.909 $\rightarrow$ 0.964 relative to configuration E3). For EfficientNetV2B0, E5 improved Over-ripe precision from 0.857 to 0.920 relative to E3 while maintaining a recall of 0.958. Consequently, the Over-ripe F_1 -score increased from 0.923 (E3) to 0.939 (E5). These observations suggest that incorporating the geometric augmentation stage contributes to improving minority-class precision while preserving high recall for the evaluated EfficientNet architectures.

4.3. Effect of Intra-Class Mixing and Spatial Transformations

Experiments E3, E4, and E5 provide insight into the individual contribution of each augmentation component. Configuration E3 (standard YOCO baseline) improved Over-ripe recall over the no-augmentation baseline for most models. These results suggest that half-image augmentation benefits minority-class detection. However, E3 achieved this at the cost of reduced Over-ripe precision in some models. For instance, YOLO26n-cls reduced the precision from 0.920 to 0.857. The results also show that E3 did not consistently improve majority-class performance. Configuration E4 (intra-class mixing alone) offered improved class-aware diversity but occasionally underperformed E3, especially for YOLO26n-cls where the Over-ripe F_1 -score decreased to 0.917. This observation suggests that the stitching boundary in E4 composites may introduce a spurious spatial cue that the model partially relies on instead of learning genuine ripeness characteristics.

In contrast, the proposed augmentation E5 consistently avoided this performance degradation across the evaluated models. A possible explanation is that applying geometric warping after the intra-class mixing process rotates and rescales the stitching seam, reducing the likelihood that the model relies on its fixed position. This encourages the model to place greater emphasis on morphological and color features associated with fruit ripeness. Consistent with this interpretation, E5 achieved higher minority-class performance than both E3 and E4 while maintaining competitive majority-class performance across the evaluated models.

4.4. Influence of Architectural Capacity on Augmentation Strategy

MobileNetV2 is the only model for which E2 (CutMix) achieved higher observed performance than the proposed E5 (92.0% vs. 89.6%, macro- F_1 0.927 vs. 0.903). A possible explanation is that this behavior is related to the interaction between MobileNetV2's inverted residual block structure and the spatial composites generated by the intra-class YOCO mixing stage. Within each inverted residual block, a depthwise convolution operates on each channel independently without cross-channel spatial interaction [34]. Consequently, the network may be less effective at integrating information across the two spatial halves of a YOCO composite within a single convolutional operation. As a result, the stitching boundary may be represented as a channel-wise discontinuity rather than a coherent spatial feature.

Despite the higher overall accuracy achieved by CutMix, E5 still maintained competitive performance for the Over-ripe minority class. The results of MobileNetV2 show that the Over-ripe F_1 -score under E5 (0.939) differed from E2 (0.960) by only 0.021, whereas the majority-class gains under E2 were larger. These observations suggest that the relative performance of the augmentation strategies depends, at least in part, on the underlying network architecture, although further investigation is required to confirm the proposed explanation.

4.5. Error Analysis and Practical Implications

The confusion matrices in Section 3.3 show similar error patterns across all six models. These observations suggest that the remaining misclassifications are primarily associated

with visual ambiguity in the dataset rather than a particular network architecture. Most errors occur between Ripe and Over-ripe bunches, where lighting, shadows, and oblique viewing angles may obscure subtle ripeness cues. A second error pattern occurs between Unripe and Under-ripe bunches, which may be associated with small bunch size and dense fruitlet packing, reducing visible color differences at the 224×224 input resolution. Although the proposed augmentation substantially improved overall classification performance and minority-class recognition, these persistent confusions indicate that the current system is not free from classification errors under practical operating conditions. In particular, visually adjacent ripeness stages remain challenging to distinguish under natural field conditions. These results suggest that additional improvements in image acquisition and model design are still required for robust real-world deployment.

These observations have several practical implications for potential mill deployment. First, because all YOLO models under E5 correctly identified every Over-ripe sample and most remaining errors occurred near the Ripe class boundary, the system could potentially employ a lower decision threshold for the Over-ripe class. This would be expected to increase false positives slightly while further reducing the likelihood of missing Over-ripe bunches. Depending on operational requirements, such a trade-off may be acceptable because undetected Over-ripe bunches can reduce crude palm oil quality. Second, the sensitivity between the Ripe and Over-ripe classes to lighting conditions suggests that controlled lighting at the mill reception ramp may help reduce these errors without retraining the model. Third, confusion between Unripe and Under-ripe samples may also be reduced by increasing the input resolution, which could preserve finer color and texture details for distinguishing these visually similar classes.

4.6. Comparison with Related Works

A comparison of the proposed work with representative prior studies on oil palm FFB ripeness classification was conducted. All selected studies utilize deep learning methodologies and provide quantitative performance metrics. Because the compared studies differ in dataset source, image acquisition conditions, number of ripeness classes, and evaluation protocols, the comparison should be interpreted as an indirect cross-study comparison rather than a direct benchmark. Three primary evaluation criteria were analyzed: the number of ripeness classes (2–5), the dataset collection conditions (laboratory vs. field), and the reported performance metrics (accuracy, macro- F_1 , or mAP).

4.6.1. Comparison with CNN-Based Classification

Table 16 presents an indirect comparison between the proposed work and previous CNN-based methods for oil palm FFB ripeness classification.

Table 16. Comparison with prior works on lightweight CNN-based oil palm FFB ripeness classification architectures.

Study	Model	Dataset Source	Classes	Accuracy (%)
[10]	MobileNetV2	Mobile-captured (Field)	4	81.0
[35]	MobileNetV2	Field Imagery	6	81.0
[11]	MobileNetV3-Small	Public Repository	4	91.5
[10]	EfficientNetB0	Mobile-captured (Field)	4	89.0
[38]	EfficientDet	Plantation Field Imagery	4	84.0
[45]	DenseNet	Studio Imagery	4	85.0
[46]	ANN	Extracted Color/Texture	3	98.3
Our work (E5)	MobileNetV2	Field Collection	4	89.6
Our work (E5)	EfficientNetV2B0	Field Collection	4	94.8
Our work (E5)	EfficientNetV2B1	Field Collection	4	94.8

The reported accuracies range from 81.0% to 98.3%, reflecting differences in model architecture, dataset source, image acquisition conditions, and the number of ripeness classes. The ANN-based method presented in [46] achieved the highest reported accuracy (98.3%) among the studies included in the comparison. However, this method used handcrafted feature-based FFB inputs and was evaluated on a three-class dataset.

Overall, the proposed architectures achieved competitive accuracy compared with previous lightweight CNN-based approaches. The proposed MobileNetV2 achieved an accuracy of 89.6% on the four-class dataset, which is higher than the MobileNetV2 result reported in [10]. The proposed MobileNetV2 model also achieved a higher reported accuracy than [35]. However, this comparison should be interpreted with caution because the latter study was conducted on a six-class dataset. Furthermore, the proposed MobileNetV2 achieved an accuracy within 1 percentage point of the MobileNetV3-Small model reported in [11].

The proposed EfficientNet-based models achieved an accuracy of 94.8%, compared with 89.0% reported for EfficientNetB0 in [10], and were within 3.5 percentage points of the highest reported accuracy among the studies considered. Compared with the EfficientDet-based approach in [38], the proposed EfficientNet models achieved a numerical improvement of 10.8 percentage points. The proposed EfficientNet models also achieved higher reported accuracy than the DenseNet-based approach presented in [45]. Although these comparisons involve different datasets and experimental settings, they indicate that the proposed framework is competitive with existing lightweight CNN-based methods for oil palm FFB ripeness classification.

4.6.2. Comparison with YOLO-Based Methods

We compared our proposed YOLO-based framework with previous studies using YOLO variants for oil palm ripeness assessment, as summarized in Table 17. The studies reviewed include both detection and classification tasks. Therefore, the reported results should be interpreted with caution, as detection-based metrics such as mAP and detection-level F_1 evaluate both fruit bunch localization and ripeness classification, whereas the proposed YOLO-cls models evaluate only image-level ripeness classification.

Table 17. Comparison with prior works on YOLO-based oil palm FFB ripeness detection and classification architectures. N/R = not reported.

Study	Model	Task	Classes	Reported Result
[17]	YOLOv3-tiny	Classification	4	$F_1 = \text{N/R}$, Accuracy = 94.37%
[39]	YOLOv4	Detection	3	$F_1 = 0.90$, mAP = 99.17%
[15]	YOLOv4	Detection	6	$F_1 = 0.99$, mAP = 99.98%
[15]	YOLOv4-tiny	Detection	6	$F_1 = 0.53$, mAP = 56.1%
[11]	YOLOv8m	Detection	4	$F_1 = 0.98$, Accuracy = 98%
[47]	YOLOv8n	Detection	4	$F_1 = 0.86$, mAP = 89.7%
[16]	YOLOv8n-cls	Classification	6	$F_1 = \text{N/R}$, Accuracy = 95.7%
[13]	YOLOv11n	Detection	5	$F_1 = 0.73$, mAP = 77.8%
Our work (E5)	YOLOv8n-cls	Classification	4	$F_1 = 0.96$, Accuracy = 95.6%
Our work (E5)	YOLO11n-cls	Classification	4	$F_1 = 0.93$, Accuracy = 92.4%
Our work (E5)	YOLO26n-cls	Classification	4	$F_1 = 0.93$, Accuracy = 92.8%

The comparison shows that the legacy YOLOv4 model achieved high object detection performance, reporting an F_1 -score of 0.99 and a mean Average Precision (mAP) of 99.98% across a six-class classification framework [15]. Similarly, another YOLOv4-based implementation reported an F_1 -score of 0.90 and an mAP of 99.17% for a three-class dataset [39]. However, the YOLOv4-tiny variant exhibited lower performance, with an F_1 -score of 0.53

and an mAP of 56.1% [15]. These results suggest that reducing the model capacity may substantially affect performance when the network is required to perform both object localization and ripeness classification simultaneously.

YOLOv8-based models have also reported strong object detection performance. The YOLOv8m model achieved an F_1 -score of 0.98 in [11]. Moreover, YOLOv8n achieved an F_1 -score of 0.86 on a four-class dataset [47]. Within the studies considered, the lightweight YOLOv8 variants maintained competitive detection performance relative to the larger YOLO architectures. Meanwhile, the newer YOLO11 architecture reported an F_1 -score of 0.73 [13]. However, this comparison should be interpreted with caution because the YOLO11 model was evaluated on a more challenging five-class FFB dataset.

Within the classification-based paradigm, the proposed YOLO configurations achieved competitive performance compared with previous studies. The proposed YOLO11n-cla and YOLO26n-cla architectures achieved accuracies of 92.4% and 92.8%, respectively, with both obtaining an F_1 -score of 0.93. The highest observed result in the present study was achieved by YOLOv8n-cla, reaching an accuracy of 95.6% and an F_1 -score of 0.96. This accuracy is higher than the YOLOv3-tiny result reported in [17], although the comparison is indirect because the studies were conducted under different experimental settings. Furthermore, the proposed YOLOv8n-cla achieved an accuracy comparable to that reported for the same backbone architecture in [16]. However, the latter study evaluated its model on a six-class dataset, and therefore the comparison should be interpreted as indicative rather than a direct benchmark.

4.7. Limitations

Several limitations should be noted. First, the dataset contains only 857 images collected from a single oil palm reception ramp in Thailand. As a result, the validation and test sets include relatively few samples per class, especially for the minority Over-ripe class. Therefore, the performance reported for this class should be interpreted with appropriate caution. In addition, data collection was restricted to one geographic location over a one-week period. Since oil palm appearance may vary across plantation regions, cultivation practices, harvest seasons, cultivars, soil conditions, and local climates, models trained on this dataset may not generalize directly to other locations without additional local data or fine-tuning. Although the proposed augmentation substantially improves classification performance, the remaining confusion between adjacent ripeness stages indicates that additional improvements are still required before the system can be considered fully robust for practical mill deployment.

Second, annotation was performed by three trained grading experts from the collection site. Although their practical experience supported the labeling process, using only three annotators from one location may have introduced site-specific bias. Furthermore, inter-annotator agreement was not quantitatively evaluated using metrics such as Cohen's Kappa because the original independent annotation records from each evaluator were not retained after the consensus labeling process. Consequently, annotation reliability could not be quantitatively validated, representing a limitation of the present study. Third, this study did not perform formal statistical significance testing to compare the evaluated augmentation strategies and model architectures. Therefore, although the proposed E5 configuration achieved the highest observed performance in most experiments, the reported differences should be interpreted as empirical observations under the evaluated experimental protocol rather than statistically confirmed evidence of superiority. Finally, although model size and inference latency were evaluated to characterize computational efficiency, latency was measured only on a desktop-class NVIDIA RTX 3080 GPU. Therefore, the reported inference times should be interpreted as relative comparisons under identical hardware conditions

rather than as expected deployment latency on embedded edge devices. Experimental validation on representative embedded platforms remains necessary to assess real-time performance, memory usage, and energy consumption under practical mill conditions.

In conclusion, these limitations indicate that the reported findings should be interpreted as evidence of feasibility under the present dataset, annotation procedure, hardware platform, and experimental protocol, rather than as definitive evidence of statistical superiority, broad geographic generalizability, quantitatively validated annotation reliability, or embedded-device deployment performance.

4.8. Future Research Directions

Based on these findings and limitations, three main directions are identified for future research. First, the proposed augmentation strategy should be evaluated on larger datasets collected from multiple plantations, regions, and harvest seasons. Future studies should also include repeated experimental trials and statistical significance analysis to further validate the observed performance differences. A more diverse group of independent annotators should be involved, and the original annotation records should be preserved so that inter-annotator agreement metrics, such as Cohen's Kappa, can be quantitatively reported prior to consensus labeling. Second, increasing the input resolution or using multi-angle images may help reduce confusion between Unripe and Under-ripe bunches. Finally, the proposed lightweight models should be implemented and benchmarked on representative embedded edge platforms to assess inference latency, memory usage, energy consumption, and real-time deployment viability under practical mill reception conditions.

5. Conclusions

This work presents an automated oil palm fresh fruit bunch (FFB) ripeness classification using lightweight deep learning models. We collected a four-class (Over-ripe, Ripe, Under-ripe, Unripe) oil palm FFB dataset directly at an operate mill reception ramp in south Thailand under unconstrained field conditions. An intra-class half-mixing augmentation framework combined with geometric transformation is proposed to resolve the class imbalance inherent in real-world FFB dataset. Six lightweight backbone architectures were implemented for FFB ripeness classification.

The experimental results show that YOLOv8n-cls achieved the highest observed overall accuracy of 95.6% with a macro- F_1 of 0.955, corresponding to a 12.4 percentage-point improvement over the no-augmentation baseline. In addition, other YOLO variants including YOLO11n-cls and YOLO26n-cls were conducted for the classification task and achieved accuracies of 92.4% and 92.8% respectively. CNN-based models, including EfficientNetV2B0, EfficientNetV2B1, and MobileNetV2, were implemented, obtaining accuracy scores of 94.8%, 94.8%, and 89.6%, respectively. These results indicate that lightweight architectures can achieve promising classification performance when combined with the proposed augmentation pipeline. The proposed method also produced an observed Over-ripe recall of 1.00 for all YOLO-family models and the highest observed F_1 -scores across the four ripeness classes. Overall, these results suggest that the proposed method may improve minority-class detection while maintaining competitive majority-class performance. However, further validation using larger multi-location datasets, repeated experiments, formal statistical significance testing, and representative embedded hardware is required before drawing broader conclusions regarding model superiority, generalizability, and practical deployment ability.

Author Contributions: Conceptualization, H.M. and F.E.N.; methodology, H.M.; software, H.M.; validation, H.M. and F.E.N.; formal analysis, H.M. and F.E.N.; investigation, H.M.; resources, H.M.; data curation, H.M.; writing—original draft preparation, H.M.; writing—review and editing, H.M. and F.E.N.; visualization, H.M.; supervision, K.H.; project administration, K.H.; funding acquisition, H.M. and K.H. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by Walailak University under the New Researcher Development scheme (Contract Number WU69238).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The complete image dataset and class annotations used in this study are publicly available in a Kaggle repository at <https://www.kaggle.com/datasets/hadeemadadum/oil-palm-fruit-ripeness-classification-dataset> (accessed on 16 July 2026).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Production: Palm Oil. Available online: <https://www.fas.usda.gov/data/production/4243000> (accessed on 23 May 2026).
2. Eya'a, N.C.; Nsimi, M.A.; Ndele, P.H.; Ngalle, B.H.; Molo, T.; Mbo, N.L.F.; Fouman, A.; Likeng, L.G.B.; Ngando, E.G.F.; Bell, J.M. A review of main factors involved in the maturation of oil palm (*Elaeis Guineensis* Jacq.) Fruit Bunches. *Am. J. Plant Sci.* **2023**, *14*, 727–741. [\[CrossRef\]](#)
3. Rangkuti, I.U.P.; Syukri, M.; Elisabeth, J.; Sari, D. Oil extraction and quality stability of crude palm oil derived from variations in palm fruit ripeness. *J. Tek. Pertan. Lampung* **2025**, *14*, 1573–1581. [\[CrossRef\]](#)
4. Goh, J.Y.; Md Yunos, Y.; Mohamed Ali, M.S. Fresh fruit bunch ripeness classification methods: A review. *Food Bioprocess Technol.* **2025**, *18*, 183–206. [\[CrossRef\]](#)
5. Chuquimarca, L.; Vintimilla, B.; Velastin, S. Banana ripeness level classification using a simple CNN model trained with real and synthetic datasets. In *Proceedings of the 18th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP 2023)*; VISAPP: Lisbon, Portugal, 2023; Volume 5, pp. 536–543. [\[CrossRef\]](#)
6. Mputu, H.S.; Abdel-Mawgood, A.; Shimada, A.; Sayed, M.S. Tomato quality classification based on transfer learning feature extraction and machine learning algorithm classifiers. *IEEE Access* **2024**, *12*, 8283–8295. [\[CrossRef\]](#)
7. Rizzo, M.; Marcuzzo, M.; Zangari, A.; Gasparetto, A.; Albarelli, A. Fruit ripeness classification: A survey. *Artif. Intell. Agric.* **2023**, *7*, 44–57. [\[CrossRef\]](#)
8. Lai, J.W.; Ramli, H.R.H.; Ismail, L.I.; Wan Hasan, W.Z. Oil palm fresh fruit bunch ripeness detection methods: A systematic review. *Agriculture* **2023**, *13*, 156. [\[CrossRef\]](#)
9. Espinoza, S.; Aguilera, C.; Rojas, L.; Campos, P.G. Analysis of fruit images with deep learning: A systematic literature review and future directions. *IEEE Access* **2024**, *12*, 3837–3859. [\[CrossRef\]](#)
10. Suharjito; Elwirehardja, G.N.; Prayoga, J.S. Oil palm fresh fruit bunch ripeness classification on mobile devices using deep learning approaches. *Comput. Electron. Agric.* **2021**, *188*, 106359. [\[CrossRef\]](#)
11. Harjanto, N.F.; Dewi, N.P.A.S.P.; Iswanto, I.A. Lightweight YOLOv8n segmentation and ShuffleNetV2 transfer learning integration for oil palm fruit ripeness classification. In *Proceedings of the 8th International Conference on Information and Communications Technology (ICOIACT)*; IEEE: New York, NY, USA, 2025; pp. 196–201.
12. Nurahman, N.; Minarni, M.; Aziz, A.; Winarti, L.; Mashami, E.; Prabowo, D.W. Palm fruit ripeness classification using BorneoNet for improved accuracy in precision agriculture. *Edelweiss Appl. Sci. Technol.* **2025**, *9*, 1389–1401. [\[CrossRef\]](#)
13. Suparto, A.; Pribadi, M.R. Improving oil palm fruit detection under class imbalance using class-balanced focal loss on YOLOv11. *J. Sisfokom* **2026**, *15*, 165–176. [\[CrossRef\]](#)
14. Redmon, J.; Divvala, S.; Girshick, R.; Farhadi, A. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016*; IEEE: New York, NY, USA, 2016; pp. 779–788.
15. Suharjito, M.; Asrol, M.; Utama, D.N.; Adeta Junior, F.; Marimin. Real-time oil palm fruit grading system using smartphone and modified YOLOv4. *IEEE Access* **2023**, *11*, 59758–59773. [\[CrossRef\]](#)
16. Josdaan, J.; Tamsil, V.C.; Harefa, J.; Jingga, K. Revolutionizing palm oil ripeness classification: Utilizing YOLOv8 for ultra-precise ripeness detection. *Procedia Comput. Sci.* **2024**, *245*, 700–709. [\[CrossRef\]](#)

17. Junos, M.H.; Mohd Khairuddin, A.S.; Abu Talip, M.S.; Kairi, M.I.; Siran, Y.M. Improved hybrid feature extractor in lightweight convolutional neural network for postharvesting technology: Automated oil palm fruit grading. *Neural Comput. Appl.* **2024**, *36*, 20473–20491. [\[CrossRef\]](#)
18. Li, J.; Zhang, T.; Zeng, S.; Gao, Q.; Wang, L.; Lu, J. StarNet-embedded efficient network for on-tree palm fruit ripeness identification in complex environments. *Agriculture* **2025**, *15*, 1823. [\[CrossRef\]](#)
19. Ruswanto, A.; Ramelan, A.H.; Praseptianga, D.; Partha, I.B.B. Effects of ripening level and processing delay on the characteristics of oil palm fruit bunches. *Int. J. Adv. Sci. Eng. Inf. Technol.* **2020**, *10*, 389–394. [\[CrossRef\]](#)
20. Shorten, C.; Khoshgoftaar, T.M. A survey on image data augmentation for deep learning. *J. Big Data* **2019**, *6*, 60. [\[CrossRef\]](#)
21. Worasawate, D.; Sakunasinha, P.; Chiangga, S. Automatic classification of the ripeness stage of mango fruit using a machine learning approach. *AgriEngineering* **2022**, *4*, 32–47. [\[CrossRef\]](#)
22. Bird, J.J.; Barnes, C.M.; Manso, L.J.; Ekárt, A.; Faria, D.R. Fruit quality and defect image classification with conditional GAN data augmentation. *Sci. Hortic.* **2022**, *293*, 110684. [\[CrossRef\]](#)
23. Miftahushudur, T.; Sahin, H.M.; Grieve, B.; Yin, H. A survey of methods for addressing imbalanced data problems in agricultural applications. *Remote Sens.* **2025**, *17*, 454. [\[CrossRef\]](#)
24. Taylor, L.; Nitschke, G. Improving deep learning with generic data augmentation. In *Proceedings of the IEEE Symposium Series on Computational Intelligence (SSCI)*; IEEE: New York, NY, USA, 2018; pp. 1542–1547.
25. Sivamani, S.; Chon, S.I.; Park, J.H. Selective augmentation to create a balanced dataset based on photometric variation. *IEEE Access* **2024**, *12*, 127460–127469. [\[CrossRef\]](#)
26. Yoo, J.; Kang, S. Class-adaptive data augmentation for image classification. *IEEE Access* **2023**, *11*, 26393–26402. [\[CrossRef\]](#)
27. Gao, X.; Jamil, N.; Ramli, M.I. CL-SR: Boosting imbalanced image classification with contrastive learning and synthetic minority oversampling technique based on rough set theory integration. *Appl. Sci.* **2024**, *14*, 11093. [\[CrossRef\]](#)
28. Shumilo, L.; Okhrimenko, A.; Kussul, N.; Drozd, S.; Shkalikov, O. Generative adversarial network augmentation for solving the training data imbalance problem in crop classification. *Remote Sens. Lett.* **2023**, *14*, 1129–1138. [\[CrossRef\]](#)
29. Yun, S.; Han, D.; Oh, S.J.; Chun, S.; Choe, J.; Yoo, Y. CutMix: Regularization strategy to train strong classifiers with localizable features. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, Seoul, Republic of Korea, 27 October–2 November 2019; IEEE: New York, NY, USA, 2019; pp. 6023–6032.
30. Walawalkar, D.; Shen, Z.; Liu, Z.; Savvides, M. Attentive CutMix: An enhanced data augmentation approach for deep learning based image classification. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Barcelona, Spain, 4–8 May 2020; IEEE: New York, NY, USA, 2020; pp. 3642–3646. [\[CrossRef\]](#)
31. Zhang, H.; Cisse, M.; Dauphin, Y.N.; Lopez-Paz, D. mixup: Beyond empirical risk minimization. In *Proceedings of the International Conference on Learning Representations (ICLR)*, Vancouver, BC, Canada, 30 April–3 May 2018; ICLR: Appleton, WI, USA, 2018.
32. Koh, G.; Oh, H.J.; Noh, J.; Jeong, W.K. Synthetic data augmentation using pre-trained diffusion models for long-tailed food image classification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: New York, NY, USA, 2025; pp. 391–400.
33. Han, J.; Fang, P.; Li, W.; Hong, J.; Armin, M.A.; Reid, I.; Petersson, L.; Li, H. You only cut once: Boosting data augmentation with a single cut. In *Proceedings of the International Conference on Machine Learning (ICML)*; PMLR: Cambridge, MA, USA, 2022; pp. 8196–8212.
34. Sandler, M.; Howard, A.; Zhu, M.; Zhmoginov, A.; Chen, L.C. MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, USA, 18–23 June 2018; IEEE: New York, NY, USA, 2018; pp. 4510–4520.
35. Sonata, I.; Arifin, Y. Deep learning approach for palm fruit ripeness classification using MobileNet. In *Proceedings of the 4th International Conference on Creative Communication and Innovative Technology (ICCIT)*; IEEE: New York, NY, USA, 2025; pp. 1–7.
36. Tan, M.; Le, Q. EfficientNetV2: Smaller models and faster training. In *Proceedings of the International Conference on Machine Learning (ICML)*; PMLR: Cambridge, MA, USA, 2021; pp. 10096–10106.
37. Yunita, F.; Usman, U.; Sawitri, N. Optimization of the EfficientNet-Swin Transformer hybrid model with dynamic attention and multi-task learning for accurate palm fruit classification. *Int. J. Intell. Eng. Syst.* **2026**, *19*, 84–92. [\[CrossRef\]](#)
38. Soetrisno, Y.A.A.; Handoyo, E.; Sinuraya, E.W. Oil palm level of ripeness classification using EfficientDet-Lite CNN architecture. *J. Oil Palm Res.* **2024**, *36*, 618–629.
39. Sopian, A.; Seminar, K.; Sudradjat, S. System detection ripeness of fresh fruits bunch palm oil with YOLOv4. In *Proceedings of the 12th Annual International Conference on Industrial Engineering and Operations Management*; IEOM: Southfield, MI, USA, 2022.
40. Ma, N.; Zhang, X.; Zheng, H.T.; Sun, J. ShuffleNetV2: Practical guidelines for efficient CNN architecture design. In *Proceedings of the European Conference on Computer Vision (ECCV)*, Munich, Germany, 8–14 September 2018; Springer: Berlin/Heidelberg, Germany, 2018; pp. 116–131.
41. Sohan, M.; Sai Ram, T.; Rami Reddy, C.V. A review on YOLOv8 and its advancements. In *Proceedings of the International Conference on Data Intelligence and Cognitive Informatics*; Springer: Berlin/Heidelberg, Germany, 2024; pp. 529–545.

42. Ali, M.L.; Zhang, Z. The YOLO Framework: A comprehensive review of evolution, applications, and benchmarks in object detection. *Computers* **2024**, *13*, 336. [\[CrossRef\]](#)
43. Sapkota, R.; Cheppally, R.H.; Sharda, A.; Karkee, M. YOLO26: Key architectural enhancements and performance benchmarking for real-time object detection. *arXiv* **2025**, arXiv:2509.25164.
44. Loshchilov, I.; Hutter, F. Decoupled weight decay regularization. In *Proceedings of the International Conference on Learning Representations (ICLR), New Orleans, LA, USA, 6–9 May 2019*; ICLR: Appleton, WI, USA, 2019.
45. Herman, H.; Cenggoro, T.W.; Susanto, A.; Pardamean, B. Deep learning for oil palm fruit ripeness classification with DenseNet. In *Proceedings of the International Conference on Information Management and Technology (ICIMTech)*; ACM Digital Library: New York, NY, USA, 2021; Volume 1, pp. 116–119.
46. Septiarini, A.; Sunyoto, A.; Hamdani, H.; Kasim, A.A.; Utaminingrum, F.; Hatta, H.R. Machine vision for the maturity classification of oil palm fresh fruit bunches based on color and texture features. *Sci. Hortic.* **2021**, *286*, 110245. [\[CrossRef\]](#)
47. Chang, C.; Parthiban, R.; Kalavally, V.; Hung, Y.M.; Wang, X. Unharvested palm fruit bunch ripeness detection with hybrid color correction. *Smart Agric. Technol.* **2024**, *9*, 100643. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.